

ECOM EXPERTS

Ecom autobarcodes

Manual de usuario, técnico y de desarrollo

ecom_autobarcodes 1.0.0 · 07/09/2026

Contenido

1. Qué resuelve

2. Instalación

3. Configuración

Prefijo EAN (6 cifras) — obligatorio

Códigos por lote

Activar depuración

4. Panel de estado

5. Generar los códigos

Opción A — Todo el catálogo

Opción B — Solo los productos que elijas

Durante el proceso

6. Preguntas frecuentes

7. Recomendación antes de la primera vez

1. Qué hace

2. Estructura de ficheros

3. Hooks registrados

3.1 El listado de productos en 1.7.8 NO es un Grid

Matriz de compatibilidad del listado

4. Cómo se integra en el listado de productos

4.1 Botón del desplegable de cabecera

4.2 Entrada en acciones en lote

5. Endpoint AJAX

Adaptaciones obligatorias para PrestaShop 9

6. Capa de datos

Dónde vive el EAN

Escritura

Validación del core

7. Algoritmo de asignación

8. Base de datos propia

9. Configuración

10. Seguridad

11. Multitienda

12. Verificación realizada

1. Decisiones de arquitectura y por qué

1.1 Nada de Symfony: sin src/, sin rutas, sin Composer

1.2 Un solo código para PS 8 y PS 9

1.3 SQL directo en vez de ObjectModel

2. Mapa de clases

3. Flujo completo de una ejecución

4. Puntos delicados

4.1 La entrada de acciones en lote se pierde al filtrar

4.2 Detección de la página

4.3 Prevención de bucle infinito en el cliente

4.4 Agotamiento del prefijo

4.5 La tabla ps_tab es crítica en PS 9

5. Cómo extender

Añadir un campo nuevo a la configuración

Cambiar la longitud del prefijo

Generar también UPC o ISBN

Añadir una migración de esquema

6. Depuración

Síntomas frecuentes

Consultas útiles

7. Cómo re-verificar tras un cambio

8. Lo que NO hace este módulo

Genera automáticamente los códigos EAN-13 que faltan en tus productos y en sus combinaciones.

1. Qué resuelve

Si tienes productos sin código de barras y los necesitas para el almacén, el TPV, un marketplace o el feed de Google Shopping, este módulo te los crea todos de golpe.

- Genera un código para el **producto base** y otro **para cada combinación**.
- Todos empiezan por **tu prefijo de 6 cifras**.
- Los códigos son **EAN-13 válidos de verdad**, con su dígito de control correcto.
- **Nunca toca un código que ya exista**. Solo rellena huecos.

2. Instalación

1. Sube el ZIP del módulo desde **Módulos** → **Gestor de módulos** → **Subir un módulo**.
2. Pulsa **Instalar**.
3. Pulsa **Configurar**.

Compatible con PrestaShop 8.x y 9.x.

3. Configuración

En la pantalla de configuración tienes tres ajustes:

Prefijo EAN (6 cifras) — obligatorio

Los 6 primeros dígitos de todos los códigos que se generen. Por ejemplo **840123**.

 **Importante:** si vas a vender fuera de tu propia tienda (marketplaces, distribuidores, grandes superficies), el prefijo debe ser un **prefijo GS1 asignado a tu empresa**. Si te lo inventas, el código funcionará dentro de tu almacén pero puede chocar con el de otro fabricante fuera. Para uso interno, cualquier prefijo vale.

Hasta que no guardes un prefijo válido de 6 cifras, el botón del listado de productos no generará nada.

Códigos por lote

Cuántos códigos se generan en cada paso. Por defecto **200**.

Si tu servidor es lento o el proceso se corta a medias, bájalo a **50**. Si tienes un catálogo enorme y un servidor potente, puedes subirlo.

Activar depuración

Déjalo en **No** salvo que estés investigando un problema. Si lo pones en **Sí**, el módulo guarda un registro detallado dentro de su carpeta **logs**, que puedes vaciar desde la propia pantalla de configuración.

4. Panel de estado

Nada más entrar en la configuración verás un resumen:

Dato	Significado
Prefijo configurado	El prefijo activo, o un aviso si falta
Productos sin EAN	Cuántos productos base están pendientes
Combinaciones sin EAN	Cuántas combinaciones están pendientes
Último número de serie usado	Por dónde va el contador
Códigos disponibles con este prefijo	Cuántos quedan libres (cada prefijo da 1.000.000)
Próximo código que se asignaría	El siguiente código exacto

5. Generar los códigos

Ve a **Catálogo** → **Productos**. Hay dos formas.

Opción A — Todo el catálogo

1. Arriba a la derecha de la lista, pulsa el **icono de ajustes** (la ruedecita del encabezado).
2. Elige **Generar EAN faltantes**.
3. Confirma.

Se recorre todo el catálogo y se asigna un código a cada producto y combinación que no tenga uno.

Opción B — Solo los productos que elijas

1. **Marca la casilla** de los productos que quieras.
2. Abre el desplegable **Acciones en lote** (abajo, se activa al marcar algo).
3. Elige **Generar EAN faltantes (seleccionados)**.
4. Confirma.

Se generan códigos para esos productos **y para todas sus combinaciones**.

Durante el proceso

Aparece una ventana con el progreso. En catálogos grandes puede tardar: el módulo trabaja por tandas para no saturar el servidor.

No cierres ni recargues la página hasta que termine. Si lo haces, no pasa nada grave: los códigos ya asignados se quedan y puedes volver a lanzarlo — continuará por donde iba.

Al acabar verás cuántos productos y cuántas combinaciones se han procesado. Pulsa el botón para cerrar y recargar la lista.

6. Preguntas frecuentes

¿Puede machacarme los EAN que ya tengo puestos? No. El módulo solo escribe donde el campo está vacío. Es una garantía del propio funcionamiento interno, no una opción que se pueda desactivar por error.

¿Puedo repetir el proceso las veces que quiera? Sí. Cada pasada solo rellena lo que falte. Si no falta nada, te avisa de que no había nada que hacer.

¿Qué pasa si cambio el prefijo más adelante? Los códigos ya generados se quedan como están. Los nuevos empezarán por el prefijo nuevo, desde el número 1. Nunca se repite un código ya usado.

Me dice que el prefijo se ha quedado sin números. Has agotado el millón de códigos de ese prefijo. Configura un prefijo distinto y sigue generando.

¿Los códigos son válidos para escanear? Sí, son EAN-13 correctos con dígito de control. Cualquier lector los lee. Para que sean **únicos en el mundo** el prefijo debe ser un prefijo GS1 tuyo (ver el aviso del punto 3).

Tengo varias tiendas. ¿Puedo poner un EAN distinto en cada una? No, y no es una limitación del módulo: PrestaShop guarda el EAN una sola vez por producto, compartido por todas las tiendas. Lo que sí hace el módulo es procesar únicamente los productos asociados a la tienda en la que estés trabajando.

¿Se ejecuta solo, en segundo plano? No, nunca. Solo se genera cuando una persona pulsa el botón. No hay tareas automáticas ni cron.

¿Envía datos a algún sitio? No. El módulo no hace ninguna conexión externa. Todo se calcula y se guarda en tu propia tienda.

El botón no me aparece en el listado. Entra una vez en la configuración del módulo (**Módulos → Ecom AutoBarcode → Configurar**): al abrirla, el módulo revisa y repara su propia instalación. Vuelve a Catálogo → Productos y recarga.

Si desinstalo el módulo, ¿pierdo los códigos? No. Los EAN se guardan en la ficha de cada producto, como si los hubieras escrito a mano. Al desinstalar solo se borra la configuración del módulo y su registro interno.

7. Recomendación antes de la primera vez

Haz una **copia de seguridad de la base de datos** antes de la primera generación masiva. Es una operación segura y no sobrescribe nada, pero es un cambio sobre muchos productos a la vez y conviene tener la red puesta.

Si quieres probar sin compromiso: selecciona **2 o 3 productos** con la Opción B, comprueba el resultado en su ficha (pestaña **Detalles**, campo EAN-13) y luego lanza el catálogo completo.

Ecom AutoBarcode — Documentación técnica

Módulo: ecom_autobarcodes · **Clase:** Ecom_Autobarcodes · **Versión:** 1.0.0 **Compatibilidad:** PrestaShop 1.7.8, 8.x y 9.x · PHP 7.1 → 8.x **Autor:** Ecom Experts · **Licencia:** AFL 3.0

ps_versions_compliancy = ['min' => '1.7.8.0', 'max' => _PS_VERSION_] . El código se mantiene compatible con **PHP 7.1** (mínimo de 1.7.8): sin propiedades tipadas, sin arrow functions, sin match, sin ??=, sin str_contains.

1. Qué hace

Genera códigos **EAN-13 válidos** para los productos y combinaciones del catálogo que no tienen uno, a partir de un prefijo de 6 cifras configurable. **Nunca sobrescribe un EAN existente.**

Estructura del código generado:

```
8 4 0 1 2 3   0 0 1 2 3 4   5
└─ prefijo ─┘ └─ serie ─┘ └─ dígito de control
   6 cifras      6 cifras      1 cifra
```

- **Prefijo:** los 6 dígitos configurados en el módulo.
- **Serie:** contador correlativo, de 000000 a 999999 → **1.000.000 de códigos por prefijo.**
- **Dígito de control:** calculado según el estándar EAN-13 (suma ponderada 1-3-1-3..., $(10 - \text{suma} \% 10) \% 10$).

2. Estructura de ficheros

```
ecom_autobarcodes/
├─ ecom_autobarcodes.php           Clase principal (global namespace)
├─ config.xml
├─ logo.png
├─ index.php
├─ classes/
│   ├─ EcomAutobarcodesEan13.php   Algoritmo EAN-13 puro (sin dependencias)
│   ├─ EcomAutobarcodesGenerator.php Motor de asignación por lotes
│   └─ EcomAutobarcodesLogger.php  Depuración opcional a /logs
├─ controllers/admin/
│   └─ AdminEcomAutobarcodesController.php Endpoint AJAX (JSON)
├─ views/
│   ├─ css/ecom_autobarcodes.css
│   ├─ js/ecom_autobarcodes.js     Botón toolbar + entrada en acciones bulk
│   └─ templates/admin/status.tpl  Panel de estado en la configuración
├─ translations/
│   ├─ es-ES/ModulesEcomautobarcodesAdmin.es-ES.xlf
│   └─ en-US/ModulesEcomautobarcodesAdmin.en-US.xlf
└─ logs/                           Ficheros .log si la depuración está activa
```

No hay src/, config/services.yml, config/routes.yml, composer.json ni vendor/. La carga de clases es manual con require_once.

3. Hooks registrados

Hook	Para qué	PS 8.2	PS 9.1
<code>actionProductGridDefinitionModifier</code>	Añade la acción al desplegable de cabecera del listado	✓	✓
<code>displayBackOfficeHeader</code>	Inyecta CSS + <code><script></code> con la configuración	✓	✓
<code>actionAdminControllerSetMedia</code>	Registra el CSS por la vía nativa cuando está disponible	✓	✓

El identificador del grid de productos es `product` en **ambas** versiones (`ProductGridDefinitionFactory::GRID_ID`), por lo que el hook se llama igual: `actionProductGridDefinitionModifier`.

3.1 El listado de productos en 1.7.8 NO es un Grid

En PrestaShop **1.7.8 el listado de productos no usa el componente Grid**: lo sirve

`Admin\ProductController::catalogAction` con plantillas propias (`Admin/Product/CatalogPage/*.twig`). Por eso:

- No existe `ProductGridDefinitionFactory` ni el grid id `product` → el hook `actionProductGridDefinitionModifier` **nunca se dispara** en 1.7.8 (queda como código muerto inofensivo, solo actúa en 8.x/9.x).
- Los checkboxes de fila se llaman `bulk_action_selected_products[]` (no `product_bulk[]`).
- El menú de acciones en lote es `#product_bulk_menu` (no `.js-bulk-actions-btn`).
- El botón "generar todo" se inyecta por JS en el menú de ajustes `#catalog-tools-button` (en 8.x/9.x lo aporta el `SimpleGridAction` nativo).

El JS del módulo usa **detección por marcado** (selectores alternativos), no `version_compare`, para cubrir las tres familias con un único fichero.

Matriz de compatibilidad del listado

	1.7.8.11	8.2	9.1
Motor del listado	CatalogPage (Twig propio)	Grid	Grid
Hook <code>actionProductGridDefinitionModifier</code>	no se dispara	✓	✓
Checkbox de fila	<code>bulk_action_selected_products[]</code>	<code>product_bulk[]</code>	<code>product_bulk[]</code>
Botón acciones en lote	<code>#product_bulk_menu</code>	<code>.js-bulk-actions-btn</code>	<code>.js-bulk-actions-btn</code>
Botón "generar todo"	inyectado en <code>#catalog-tools-button</code>	<code>SimpleGridAction</code>	<code>SimpleGridAction</code>
<code>displayBackOfficeHeader</code> / <code>actionAdminControllerSetMedia</code>	✓ (vía AdminLegacyLayoutController)	✓	✓

4. Cómo se integra en el listado de productos

4.1 Botón del desplegable de cabecera

Se añade un `SimpleGridAction` a la colección de acciones del grid:

```
$definition->getGridActions()->add(
    (new SimpleGridAction('ecomautobarcodes_generate'))
    ->setName($this->trans('Generar EAN faltantes', [], $this->domain))
    ->setIcon('qr_code_2')
);
```

PrestaShop lo renderiza (`Admin/Common/Grid/Actions/Grid/simple.html.twig` , idéntico en 8.2 y 9.1) como:

```
<button type="button"
    id="product_grid_action_ecomautobarcodes_generate"
    class="dropdown-item js-ecomautobarcodes_generate-grid-action">...</button>
```

`SimpleGridAction` **no requiere ninguna ruta Symfony** — a diferencia de `LinkGridAction` y `SubmitGridAction` , cuyas plantillas llaman a `path()` . A cambio es inerte: el módulo aporta su propio listener JS sobre `.js-ecomautobarcodes_generate-grid-action` .

4.2 Entrada en acciones en lote

Las clases nativas `SubmitBulkAction` y `AjaxBulkAction` **sí** exigen una ruta Symfony (`submit_route` / `ajax_route`), así que **no se usan** . En su lugar el JS del módulo inserta un `<button class="dropdown-item">` dentro del `.dropdown-menu` contiguo a `.js-bulk-actions-btn` .

Los identificadores se leen de los checkboxes nativos del grid, cuyo marcado está confirmado en ambas versiones:

```
<input type="checkbox" name="product_bulk[]" class="js-bulk-action-checkbox" value="{id_product}">
```

El JS recoge `document.querySelectorAll('.js-bulk-action-checkbox:checked')` y envía los `value` .

Un `MutationObserver` reinserta la entrada cuando el grid se repinta al filtrar u ordenar.

5. Endpoint AJAX

`controllers/admin/AdminEcomAutobarcodesController.php` — `ModuleAdminController` legacy, plenamente soportado en PS 9.1 a través de `PrestaShopBundle\Controller\Admin\LegacyController` .

Petición:

```
POST {getAdminLink('AdminEcomAutobarcodes')}&ajax=1&action=generate_ean
Content-Type: multipart/form-data

scope=all                (todo el catálogo)
- o -
scope=selection
ids[0]=12&ids[1]=45...   (id_product seleccionados)
```

Respuesta:

```
{ "success": true, "code": "ok", "products": 200, "combinations": 0,
  "skipped": 0, "remaining": 1834, "exhausted": false }
```

Códigos de error: `forbidden`, `no_prefix`, `no_selection`, `error`.

El cliente repite la llamada hasta que `remaining` llega a 0, no hay progreso, o `exhausted` es `true`.

Adaptaciones obligatorias para PrestaShop 9

Elemento	Motivo
<code>die(json_encode(...))</code> en vez de <code>\$this->ajaxDie()</code>	<code>ajaxDie()</code> fue eliminado de <code>Controller</code> en PS 9.1 (<code>Call to undefined method</code>). Es lo que hace el propio core de PS9.
<code>\$this->trans()</code> en vez de <code>\$this->l()</code>	<code>l()</code> fue eliminado de la cadena <code>AdminController</code> en PS 9.1.
Fila en <code>ps_tab</code> obligatoria con la columna <code>module</code>	<code>LegacyRouterChecker</code> resuelve el controlador del módulo a partir de esa fila; sin ella, 404.
Deduplicado de <code>ps_tab</code> por <code>class_name</code>	<code>LegacyControllerContextBuilder::getTabId()</code> captura <code>NoResultException</code> pero no <code>NonUniqueResultException</code> : dos filas duplicadas rompen el back office entero.
Token nunca cacheado, siempre vía <code>getAdminLink()</code>	En PS 9.1 <code>Tools::getAdminToken()</code> ignora su argumento y devuelve un único token CSRF de Symfony por empleado.

6. Capa de datos

Dónde vive el EAN

Verificado sobre `db_structure.sql` de 8.2.7 y 9.1.1:

Tabla	Columna	PS 8.2	PS 9.1
<code>PREFIX_product</code>	<code>ean13</code>	<code>varchar(13)</code>	<code>varchar(20)</code>
<code>PREFIX_product_attribute</code>	<code>ean13</code>	<code>varchar(13)</code>	<code>varchar(20)</code>
<code>PREFIX_product_shop</code>	—	no existe	no existe
<code>PREFIX_product_attribute_shop</code>	—	no existe	no existe

Consecuencia: el EAN13 es **global al producto/combinación**. No es un campo multitienda y no existe forma nativa de tener un EAN distinto por tienda.

Tampoco hay índice ni restricción `UNIQUE` sobre `ean13` en ninguna de las dos versiones: el core admite duplicados sin quejarse, y la unicidad la garantiza este módulo.

Escritura

Se usa **SQL directo**, no el `ObjectModel`, por dos razones:

1. **Garantía de no sobrescritura.** La condición viaja en el `WHERE`, así que la comprobación y la escritura son una sola operación atómica:

```
sql UPDATE PREFIX_product SET ean13 = '8401230000018' WHERE id_product = 42 AND (ean13 IS NULL OR ean13 = '')
```

Si otro proceso asignó un EAN entre medias, `Affected_Rows()` devuelve 0, el módulo lo cuenta como `skipped` y sigue. Con `$product->ean13 = ...; $product->update()` habría una ventana de carrera entre leer y escribir.

2. **Rendimiento.** Cargar el `ObjectModel` completo por cada producto en un catálogo de decenas de miles es inviable.

Es la misma columna que escribe el core por debajo: la cadena CQRS de PS9 (`DetailsFiller` → `ProductRepository::partialUpdate`) acaba en `$product->ean13 = ...` sobre el `ObjectModel` legacy.

Tras cada lote se marca `product_shop.indexed = 0` y `product.indexed = 0` para que el buscador nativo reindexe (`ean13` está en la lista de campos indexados del core).

Validación del core

`Validate::isEan13()` es `!$ean13 || preg_match('/^[0-9]{0,13}$/', $ean13)` : **no exige 13 dígitos y no comprueba el dígito de control**. PS 9.1 añade `isGtin()` (`/^[0-9]{0,14}$/'`) para el producto, pero la combinación sigue en `isEan13` (13). Generando siempre EAN-13 de 13 dígitos con dígito de control correcto se cumplen ambas versiones y ambos objetos.

7. Algoritmo de asignación

1. Validar el prefijo (6 dígitos exactos).
2. Cargar en memoria las series ya ocupadas:


```
SELECT ean13 FROM product WHERE ean13 LIKE 'PREFIX0%'
SELECT ean13 FROM product_attribute WHERE ean13 LIKE 'PREFIX0%'
```

 Se extrae la serie de cada uno y se guarda en un mapa.
3. Serie inicial = `ECOM_AUTOBARCODE_LAST_SERIAL + 1`.
4. Por cada producto/combinación pendiente:
 - reservar la siguiente serie libre (saltando las del mapa)
 - construir el EAN-13
 - UPDATE con el filtro de no sobrescritura
 - registrar en `ecom_autobarcodes_log`
5. Guardar el contador y marcar los productos para reindexar.
6. Devolver cuántos quedan pendientes.

El paso 2 hace que el contador sea **auto-reparable**: aunque `LAST_SERIAL` se pierda o se reinicie, las series ya usadas nunca se reutilizan.

Los productos base se procesan **antes** que las combinaciones dentro de cada lote.

8. Base de datos propia

```
CREATE TABLE PREFIX_ecom_autobarcodes_log (
  id_ecom_autobarcodes_log INT(11) UNSIGNED NOT NULL AUTO_INCREMENT,
  id_product                INT(11) UNSIGNED NOT NULL DEFAULT 0,
  id_product_attribute      INT(11) UNSIGNED NOT NULL DEFAULT 0,
  ean13                    VARCHAR(13) NOT NULL,
  id_employee               INT(11) UNSIGNED NOT NULL DEFAULT 0,
  date_add                  DATETIME NOT NULL,
  PRIMARY KEY (id_ecom_autobarcodes_log),
  UNIQUE KEY uniq_ean13 (ean13),
  KEY idx_target (id_product, id_product_attribute)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Tabla de trazabilidad: qué código se asignó, a qué, quién y cuándo. Se rellena con `INSERT IGNORE`.

9. Configuración

Clave	Tipo	Por defecto	Descripción
<code>ECOM_AUTOBARCODE_PREFIX</code>	string(6)	<code>''</code>	Prefijo. Validado con <code>/^[0-9]{6}\$/</code> .
<code>ECOM_AUTOBARCODE_LAST_SERIAL</code>	int	<code>0</code>	Última serie consumida. Se reinicia a 0 al cambiar de prefijo.
<code>ECOM_AUTOBARCODE_BATCH</code>	int	<code>200</code>	Códigos por petición. Acotado a <code>[10, 1000]</code> .
<code>ECOM_AUTOBARCODE_DEBUG</code>	bool	<code>0</code>	Activa el log en <code>/logs</code> .
<code>ECOM_AUTOBARCODE_DB_VERSION</code>	int	<code>1</code>	Versión de estructura aplicada.

Todas con `Configuration::getGlobalValue` / `updateGlobalValue`: son ajustes globales, coherentes con el hecho de que el EAN no es multitienda.

10. Seguridad

- `if (!defined('_PS_VERSION_')) exit;` en todos los `.php`.
- `index.php` con redirección en cada directorio, recursivo.
- `checkAccess()` exige empleado autenticado y perfil SuperAdmin o rol `ROLE_MOD_TAB_ADMINPRODUCTS_UPDATE` (`Access::isGranted`, presente en 8.2 y 9.1).
- Token de administración en la URL del endpoint, generado en cada renderizado.
- Enteros casteados con `(int)`, cadenas con `pSQL()`, identificadores con `bqSQL()`.
- Salida Smarty escapada con `|escape:'html':'UTF-8';` sin `nofilter`.
- Sin `eval`, sin código ofuscado, sin llamadas a servicios externos.

11. Multitienda

`Shop::isFeatureActive()` activa un `INNER JOIN` con `product_shop` / `product_attribute_shop` restringido a `Shop::getContextListShopID()`, de modo que "generar para todo el catálogo" afecta solo a los productos asociados a la tienda del contexto.

Aviso: como la columna `ean13` es global, el código asignado se ve desde **todas** las tiendas. El filtro decide *qué productos* se procesan, no en qué tienda se guarda el valor.

12. Verificación realizada

- Sintaxis: `php -l` sobre los 21 ficheros PHP en **PHP 7.4, 8.1 y 8.3** — sin errores.
- Validador PrestaShop local: sin errores críticos.
- Algoritmo EAN-13: batería de tests contra 4 EAN-13 reales conocidos (`4006381333931`, `5901234123457`, `9780201379624`, `8412345678905`), casos límite, y barrido de **200.000 códigos consecutivos** → 0 inválidos, 0 duplicados.
- Traducciones: script determinista de extracción y diff → 41 cadenas en código, 41 `rename` en los `.xlf`, 0 desajustes.
- API de grid, plantillas Twig y esquema SQL contrastados leyendo directamente los cores 8.2.7 y 9.1.1.

Ecom AutoBarcode — Guía profesional de desarrollo

Documento para quien tenga que **mantener, ampliar o depurar** el módulo.

1. Decisiones de arquitectura y por qué

1.1 Nada de Symfony: sin `src/`, sin rutas, sin Composer

El módulo sigue la estructura legacy: clase principal en global namespace, `classes/` con `require_once`, `controllers/admin/` con `ModuleAdminController`. **No existe** `src/`, `config/services.yml`, `config/routes.yml`, `composer.json` ni `vendor/`.

Esto obligó a resolver dos problemas que "de manual" se resolverían con una ruta Symfony:

Necesidad	Solución de manual (descartada)	Solución aplicada
Botón en el desplegable de cabecera	<code>LinkGridAction</code> → su Twig llama a <code>path(action.options.route)</code> → exige ruta Symfony	<code>SimpleGridAction</code> → renderiza un <code><button></code> sin ruta + listener JS propio
Acción en lote	<code>SubmitBulkAction</code> (<code>submit_route</code>) o <code>AjaxBulkAction</code> (<code>ajax_route</code>) → exigen ruta Symfony	Inyección DOM de un <code><button</code> <code>class="dropdown-item"></code> + lectura de los checkboxes nativos
Endpoint que ejecuta el trabajo	Controlador Symfony en <code>src/Controller/Admin/</code>	<code>ModuleAdminController</code> legacy + AJAX JSON

Contrastado leyendo las plantillas del core: `Actions/Grid/link.html.twig` y `Actions/Bulk/submit.html.twig` invocan `path()`; `Actions/Grid/simple.html.twig` **no**.

1.2 Un solo código para PS 8 y PS 9

No hay `version_compare` en ninguna parte. Se consiguió eligiendo únicamente APIs cuya firma y comportamiento son idénticos en 8.2.7 y 9.1.1, y evitando de entrada todo lo que cambió:

Prohibido en este módulo	Motivo
<code>\$this->l('...')</code>	Eliminado de la cadena <code>AdminController</code> en PS 9.1 → fatal error
<code>\$this->ajaxDie(...)</code>	Eliminado de <code>Controller</code> en PS 9.1 → <code>Call to undefined method</code>
<code>\$this->addJquery()</code>	Eliminado de <code>Controller</code> en PS 9.1
<code>Tab::getIdFromClassName()</code> para comprobar existencia	Caché estática poco fiable; se usa <code>Db::getValue</code>
Cachear el token de administración	En PS 9.1 es un único token CSRF de Symfony por empleado
<code>\$controller::class</code>	Incompatible con PHP 7.4
<code>declare(strict_types=1)</code>	Incompatible con el resto del catálogo de módulos
Sobrescribir <code>isAnonymousAllowed()</code>	Cambió de <code>protected</code> (PS8) a <code>public</code> (PS9)

1.3 SQL directo en vez de ObjectModel

Ver DOC_TECNICO §6. Resumen: la condición "no tiene EAN" viaja **dentro del WHERE**, lo que hace la comprobación y la escritura atómicas y elimina la ventana de carrera. Además evita instanciar decenas de miles de `ObjectModel`.

2. Mapa de clases

```

EcomAutobarcodesEan13      Algoritmo puro. Sin PrestaShop, sin BD. Testeable aislado.
├ checkDigit(base12)      Dígito de control EAN-13
├ build(prefix, serial)   Construye el código completo
├ isValid(ean)            Valida formato + dígito de control
├ isValidPrefix(prefix)   /^[0-9]{6}$/
└ extractSerial(ean, px)  Operación inversa de build()

EcomAutobarcodesGenerator  Motor. Única clase que toca la BD del catálogo.
├ countPending(ids?)      Cuántos faltan
├ processBatch(ids?, n)   Procesa un lote y devuelve el resultado
├ loadUsedSerials()       Inventario de series ocupadas (auto-reparación)
├ allocateEan()           Reserva la siguiente serie libre
├ assignProductEan()      UPDATE guardado
└ assignCombinationEan()  UPDATE guardado

EcomAutobarcodesLogger     Depuración opcional a /logs. No-op si está desactivada.

```

`EcomAutobarcodesEan13` no depende de PrestaShop **a propósito**: se puede cargar en un script suelto y testear. Si añades algo ahí, no metas `Tools::`, `Configuration::` ni `Db::`.

3. Flujo completo de una ejecución

```

Usuario pulsa el botón
|
▼
ecom_autobarcodes.js — confirm() → bucle de lotes
|
| POST scope=all|selection & ids[] |
|
▼
AdminEcomAutobarcodesController
checkAccess()
ajaxProcessGenerateEan()
|
▼
EcomAutobarcodesGenerator::processBatch()
loadUsedSerials()
fetchPendingProducts()      LIMIT n
fetchPendingCombinations()  LIMIT resto
allocateEan() + UPDATE guardado
persistLastSerial()
markForReindex()
countPending() → remaining
|
└── JSON ───────────────────────────┘
    remaining > 0 → otra vuelta
    remaining = 0 → resumen y recarga

```

4. Puntos delicados

4.1 La entrada de acciones en lote se pierde al filtrar

El grid se repinta por AJAX al filtrar u ordenar, y el `.dropdown-menu` se reconstruye. Por eso hay un `MutationObserver` sobre `document.body` (debounce de 150 ms) que reinserta el `<button>`. El listener de click está **delegado en** `document`, así que sobrevive a cualquier repintado.

Si en el futuro el marcado cambia, los dos selectores a revisar son:

```
document.querySelector('.js-bulk-actions-btn') // botón "Acciones en lote"
button.parentNode.querySelector('.dropdown-menu') // su menú (dentro de .btn-group)
```

4.2 Detección de la página

`isProductListPage()` busca `/sell/catalog/products` en `REQUEST_URI`. Cubre PS 8.2 (`/sell/catalog/products-v2/`) y PS 9.1 (`/sell/catalog/products/`), y también coincide con la ficha de producto — es inofensivo porque el JS no encuentra los elementos del grid y no hace nada.

4.3 Prevención de bucle infinito en el cliente

Tres frenos: - `MAX_ROUNDS = 2000` vueltas. - Si un lote devuelve `products + combinations === 0`, se corta (no hubo progreso). - Si `remaining <= 0`, se corta.

4.4 Agotamiento del prefijo

Un prefijo da 1.000.000 de códigos. Al acabarse, `allocateEan()` devuelve `false`, el lote marca `exhausted: true` y el cliente muestra el aviso. **No se recicla ninguna serie**. La solución es cambiar el prefijo.

4.5 La tabla `ps_tab` es crítica en PS 9

Sin una fila en `ps_tab` con `class_name = 'AdminEcomAutobarcodes'` y `module = 'ecom_autobarcodes'`, PS 9.1 devuelve 404 en el endpoint. Y con **dos** filas, `LegacyControllerContextBuilder::getTabId()` lanza una `NonUniqueResultException` no capturada que rompe **todo el back office**, no solo este módulo.

Por eso `fixTabIntegrity()` se invoca en `install()` y en `addnewfeatures()`, y deduplica por SQL crudo.

5. Cómo extender

Añadir un campo nuevo a la configuración

1. Constante en la clase que corresponda (`EcomAutobarcodesGenerator` o `EcomAutobarcodesLogger`).
2. `Configuration::updateGlobalValue(...)` en `install()`.
3. `Configuration::deleteByName(...)` en `uninstall()`.
4. Entrada en `renderConfigForm()`.
5. **Entrada en** `getConfigFormValues()` — si falta, Smarty lanza `Undefined array key` y la configuración devuelve un 500 en modo dev.
6. Lectura y validación en `postProcessConfig()`.

Cambiar la longitud del prefijo

Todo está parametrizado en `EcomAutobarcodesEan13`:

```
const PREFIX_LENGTH = 6;
const SERIAL_LENGTH = 6; // debe cumplir PREFIX_LENGTH + SERIAL_LENGTH = 12
const MAX_SERIAL = 999999; // 10^SERIAL_LENGTH - 1
```

Las tres deben cambiarse a la vez, y hay que actualizar el texto de la etiqueta y su traducción.

Generar también UPC o ISBN

`Product` y `Combination` tienen `upc` (12 dígitos) e `isbn`. El patrón es el mismo: duplicar `EcomAutobarcodesEan13` con su algoritmo y añadir los métodos `assignXxx()` al generador. Ojo: `upc` tampoco es multitienda.

Añadir una migración de esquema

No crear ficheros `upgrade/`. Se añade dentro de `addnewfeatures()`, envuelto en `try {}`, comprobando antes con `INFORMATION_SCHEMA` si el cambio ya está aplicado, y subiendo `DB_VERSION`.

6. Depuración

1. Configuración del módulo → **Activar depuración** = Sí.
2. Reproducir el problema.
3. Leer `modules/ecom_autobarcodes/logs/ecom_autobarcodes-AAAA-MM-DD.log`.

Se registran: carga del inventario de series, resultado de cada lote, prefijo agotado, fallos de tabla, fallos de pestaña y errores capturados.

Síntomas frecuentes

Síntoma	Causa probable
El botón no aparece en el listado	Hook <code>actionProductGridDefinitionModifier</code> no registrado → entrar en la configuración del módulo dispara <code>addnewfeatures()</code> y lo registra
El botón aparece pero no hace nada	El JS no cargó: mirar la consola y comprobar que <code>displayBackOfficeHeader</code> está registrado
404 al pulsar	Falta la fila en <code>ps_tab</code> , o el token caducó (recargar la página)
"Invalid security token"	Página abierta hace mucho: recargar
Back office entero roto tras instalar en PS9	<code>ps_tab</code> duplicada: entrar en la configuración del módulo ejecuta <code>fixTabIntegrity()</code>
Genera 0 códigos	Todos ya tienen EAN, o el prefijo no está configurado
Timeout en catálogos grandes	Bajar "Códigos por lote" a 50

Consultas útiles

```
-- ¿Cuántos faltan?
SELECT COUNT(*) FROM ps_product WHERE ean13 IS NULL OR ean13 = '';
SELECT COUNT(*) FROM ps_product_attribute WHERE ean13 IS NULL OR ean13 = '';

-- ¿Qué ha generado el módulo?
SELECT * FROM ps_ecom_autobarcodes_log ORDER BY date_add DESC;

-- ¿Hay EAN duplicados en el catálogo? (el core lo permite)
SELECT ean13, COUNT(*) c FROM ps_product
WHERE ean13 <> '' GROUP BY ean13 HAVING c > 1;
```

7. Cómo re-verificar tras un cambio

```
# Sintaxis en las tres versiones de PHP objetivo
for f in $(find . -name "*.php"); do
  for v in php74 php81 php83; do C:/dev-tools/bin/$v.cmd -l "$f"; done
done

# Validador PrestaShop
C:/dev-tools/bin/php81.cmd \
  C:/dev-tools/prestashop_skills/prestashop-validator/validator.php \
  --path="Z:/modulos ps mios finalizados y ok/modulos IA/ecom_autobarcodes"
```

Traducciones: tras tocar cualquier cadena hay que regenerar los `.xlf`. Nunca a ojo — script determinista que extrae con regex de `.php` / `.tpl` y hace diff contra los `resname` del `.xlf`. Debe dar 0 faltantes y 0 sobrantes.

Algoritmo: cualquier cambio en `EcomAutobarcodesEan13` exige volver a pasar la batería (EAN-13 reales conocidos + barrido de 200.000 códigos comprobando 0 inválidos y 0 duplicados).

8. Lo que NO hace este módulo

Documentado a propósito para evitar malentendidos:

- **No sobrescribe** ningún EAN existente, en ninguna circunstancia.
- **No borra** EAN al desinstalar.
- **No permite un EAN distinto por tienda** — la columna `ean13` es global en el core y no hay `product_shop.ean13`.
- **No valida** que el prefijo esté realmente asignado a la empresa por GS1. Si vas a vender fuera de tu tienda, usa un prefijo GS1 propio.
- **No genera** imágenes de código de barras, solo el número.
- **No se ejecuta automáticamente:** nunca hay generación en background ni en cron; siempre la lanza un empleado a mano desde el listado.
- **No hace ninguna llamada a servicios externos.**